

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 3: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 22 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** Compare the adjacency-list and adjacency-matrix representations of a graph with n vertices and m edges, in terms of (a) space, (b) time to test whether (u, v) is an edge, and (c) time to iterate over all neighbors of a vertex v . For which kinds of graphs (sparse vs. dense) is each representation preferable?
- A2.** State and prove the Handshake Lemma: for any undirected graph, $\sum_v \deg(v) = 2|E|$. Make sure your proof correctly accounts for self-loops.
- A3.** Define BFS layers $L_0, L_1, L_2, \dots$ with respect to a source s . Prove that $\text{dist}(s, v) = k$ if and only if $v \in L_k$ (i.e. prove that BFS correctly computes shortest-path distances in an unweighted graph).
- A4.** Prove that BFS (from a single source, on a graph with n vertices and m edges given as an adjacency list) runs in $O(n + m)$ time. Be precise about what is counted: how many times is each vertex enqueued, and how many times is each edge examined?
- A5.** Explain how to reconstruct an actual shortest path (not just its length) from BFS, using parent pointers. Why is it necessary to record a parent pointer only the *first* time a vertex is discovered?
- A6.** Prove that, for an undirected graph, the connected components computed by repeatedly running BFS from every unvisited vertex (in increasing order of vertex index) do not depend on the BFS starting order *within* a component – i.e. the partition of V into components is well-defined regardless of which vertex of a component is visited first.
- A7.** The *diameter* of a connected graph is the maximum shortest-path distance between any pair of vertices. Explain why computing the diameter via “run BFS from every vertex and take the overall maximum” costs $O(n(n + m))$, and why this can be too slow for very large graphs (e.g. $n = 10^6$).
- A8.** Compare recursive DFS and iterative DFS (with an explicit stack). In particular: (a) what determines the maximum recursion depth / stack size in the worst case, and on what kind of

graph is this worst case achieved? (b) Why must neighbors be pushed onto the explicit stack in *reverse* order to match the recursive visitation order?

- A9.** Define *discovery time* $d[u]$ and *finish time* $f[u]$ for a DFS. State (without proof) the “parenthesis property” relating the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ for any two vertices u, v .
- A10.** Prove that an undirected graph contains a cycle if and only if, during a DFS from some vertex, we encounter an edge (u, v) where v is already visited and $v \neq \text{parent}[u]$ (a back edge to a non-parent).
- A11.** Explain precisely why, in the adjacency list built from an undirected edge list, the edge $\{u, v\}$ appears in *both* $\text{adj}[u]$ and $\text{adj}[v]$, and why a naive cycle detector that does not exclude the parent edge would incorrectly report *every* edge as a cycle.
- A12.** Define *bipartite graph* and *2-coloring*. Describe the BFS-based algorithm for 2-coloring a graph and explain how it handles graphs with multiple connected components.
- A13.** State and prove the theorem: “A graph is bipartite if and only if it has no odd cycle.” (Both directions.)
- A14.** Suppose BFS 2-coloring discovers an edge $\{u, v\}$ with $\text{color}[u] = \text{color}[v]$, where u and v are in the same BFS tree rooted at s , with $\text{dist}(s, u) = d_u$ and $\text{dist}(s, v) = d_v$. Explain how to construct an explicit odd cycle from this edge and the two BFS-tree paths $s \rightarrow u$ and $s \rightarrow v$, and explain why its length $d_u + d_v + 1$ must be odd.
- A15.** For directed graphs, define the three DFS vertex colors (white, gray, black) and explain what each color represents at any point during the DFS.
- A16.** Prove that a directed graph has a directed cycle if and only if some DFS encounters an edge (u, v) where v is colored gray at the time the edge is examined.
- A17.** Define *topological order* of a DAG. Prove that a directed graph has a topological order if and only if it is acyclic.
- A18.** Prove that, for a DAG, reversing the order in which vertices *finish* during a DFS produces a valid topological order. Where in the proof do you use the fact that the graph has no cycle?
- A19.** Describe Kahn’s algorithm for topological sort (in-degree + queue). Explain the correspondence between “a vertex currently has in-degree 0 (among remaining vertices)” and “this vertex can legally be placed next in the topological order”. What does it mean if Kahn’s algorithm terminates having output fewer than n vertices?
- A20.** Give an example of a DAG with exactly two distinct topological orders, and an example of a DAG with a *unique* topological order (other than a graph with no edges). What structural property of the DAG guarantees a unique topological order?
- A21.** Describe the Quick-Find data structure (the array $\text{id}[]$, and the **find**/**union** operations). Prove that a sequence of $n - 1$ unions that merges n singleton sets into a single set can cost $\Theta(n^2)$ total time with Quick-Find.
- A22.** Describe Quick-Union (parent-pointer trees, no balancing). Give a sequence of $n - 1$ unions that produces a tree of height $n - 1$ (a path), and explain why a single **find** on the deepest node then costs $\Theta(n)$.

- A23.** Explain *union by rank* and *path compression* as separate optimizations to Quick-Union. (a) Prove that union by rank alone guarantees every tree has height $O(\log n)$. (b) Explain informally (no formal proof required) why path compression, combined with union by rank, makes the *amortized* cost per operation nearly constant ($O(\alpha(n))$, the inverse Ackermann function) over a long sequence of operations.

Part B: Quiz

- B1. (True/False)** In an unweighted graph, BFS from s always visits vertices in non-decreasing order of their distance from s .
- B2. (Multiple choice)** On a graph with n vertices and m edges given as an adjacency list, BFS and DFS from a single source both run in:
- (a) $O(n^2)$
 - (b) $O(n + m)$
 - (c) $O(nm)$
 - (d) $O(m \log n)$
- B3. (Short answer)** A self-loop (v, v) contributes how much to $\deg(v)$, by the convention used so the Handshake Lemma holds?
- B4. (True/False)** In an undirected graph, when DFS at vertex u examines a neighbor v such that $v = \text{parent}[u]$, this always indicates a cycle.
- B5. (Multiple choice)** A graph is bipartite if and only if it contains:
- (a) no cycle at all
 - (b) no even cycle
 - (c) no odd cycle
 - (d) no self-loop
- B6. (Short answer)** During a 3-color DFS on a directed graph, an edge (u, v) where v is *black* indicates what (cycle / no cycle)? What about an edge where v is *gray*?
- B7. (True/False)** Every DAG has at least one topological order, and if the DAG has ≥ 2 vertices with no edge between them, it has more than one topological order.
- B8. (Multiple choice)** Kahn's algorithm for topological sort is most directly analogous to:
- (a) DFS with a stack
 - (b) BFS with a queue, using in-degrees
 - (c) binary search
 - (d) Quick-Find
- B9. (Short answer)** With plain Quick-Find (no optimizations), what is the worst-case time for a single union operation on n elements? What about **find**?
- B10. (True/False)** With union by rank *and* path compression, the amortized cost per Union-Find operation over a long sequence of operations is $O(\log n)$ in the tightest known bound (as opposed to a smaller bound).

Part C: Solutions

Part A

- A1. (a) Space:** adjacency list uses $\Theta(n + m)$; adjacency matrix uses $\Theta(n^2)$ regardless of m . **(b) Edge test** $(u, v) \in E$: adjacency matrix $O(1)$ (direct lookup $A[u][v]$); adjacency list $O(\deg(u))$ (must scan $adj[u]$). **(c) Iterate neighbors of v :** adjacency list $\Theta(\deg(v))$ (only the actual neighbors are stored); adjacency matrix $\Theta(n)$ (must scan the whole row, including zeros). For **sparse** graphs ($m = O(n)$, the common case for real networks), the adjacency list's $\Theta(n + m)$ space and $\Theta(\deg(v))$ iteration are far better. For **dense** graphs ($m = \Theta(n^2)$), the adjacency matrix's $O(1)$ edge test can be worth its $\Theta(n^2)$ space, since $\Theta(n^2)$ is the same order as m anyway.
- A2. Handshake Lemma:** $\sum_v \deg(v) = 2|E|$. **Proof:** every edge $\{u, v\}$ with $u \neq v$ contributes exactly 1 to $\deg(u)$ and 1 to $\deg(v)$, for a total contribution of 2 to $\sum_v \deg(v)$. A self-loop (v, v) is defined to contribute 2 to $\deg(v)$ directly (by convention), so it also contributes 2 to the sum. Summing over all $|E|$ edges, each contributing exactly 2, gives $\sum_v \deg(v) = 2|E|$.
- A3.** Define $L_k = \{v : \text{dist}(s, v) = k\}$, where $\text{dist}(s, v)$ is the true shortest-path distance. We show by strong induction on k that BFS sets $\text{dist}[v] = k$ exactly for $v \in L_k$. *Base case* ($k = 0$): $L_0 = \{s\}$ and BFS initializes $\text{dist}[s] = 0$. *Inductive step:* assume BFS correctly computes $\text{dist}[v] = j$ for all $v \in L_j$, $j < k$, and (by the FIFO discipline of the queue) that all vertices in $L_0, \dots, L_{k-1}$ are dequeued before any vertex in L_k is enqueued. Take $v \in L_k$: by definition of distance, v has some neighbor $u \in L_{k-1}$ (the predecessor on a shortest path), and no neighbor in $L_0, \dots, L_{k-2}$ (else $\text{dist}(s, v) < k$). When u is dequeued (with $\text{dist}[u] = k - 1$ by IH), v is still unvisited (its only possible “earlier” neighbors are in L_{k-1} , processed in this same round, or L_k itself, not yet processed), so v is discovered with $\text{dist}[v] = \text{dist}[u] + 1 = k$. Conversely, if BFS sets $\text{dist}[v] = k$, it must be via a neighbor u with $\text{dist}[u] = k - 1 \in L_{k-1}$ (by IH), so v has a path of length k from s , hence $\text{dist}(s, v) \leq k$; and $v \notin L_0 \cup \dots \cup L_{k-1}$ (those vertices were already assigned distances $< k$ and would not be re-discovered), so $\text{dist}(s, v) \geq k$, giving $\text{dist}(s, v) = k$, i.e. $v \in L_k$.
- A4.** Each vertex is enqueued at most once: a vertex is enqueued only when its **dist** is first set (from ∞ to a finite value), and **dist** is never overwritten afterward. So there are at most n enqueue/dequeue operations, each $O(1)$. For each dequeued vertex u , the inner loop iterates over $adj[u]$, doing $O(1)$ work per neighbor; summing $|adj[u]|$ over all dequeued u gives $\sum_u \deg(u) = O(m)$ (each undirected edge appears in two adjacency lists, each directed edge in one). Total: $O(n) + O(m) = O(n + m)$.
- A5.** Whenever BFS discovers vertex v for the first time (sets $\text{dist}[v]$ from ∞ to a finite value) via neighbor u , record $\text{parent}[v] = u$. To reconstruct a shortest path from s to t : start at t , repeatedly move to $\text{parent}[\cdot]$ until reaching s , then reverse the resulting sequence. Recording the parent only on *first* discovery is essential because the first time a vertex is discovered is exactly when $\text{dist}[v]$ is set to its correct (minimum) value, via a neighbor at distance $\text{dist}[v] - 1$ – i.e. via an edge that lies on *some* shortest path to v . If we updated $\text{parent}[v]$ on later (non-discovering) visits, we might overwrite it with a neighbor that does *not* lie on a shortest path, breaking the reconstruction.
- A6.** The partition into connected components is determined entirely by the *reachability relation* (“ u and v are in the same component iff there is a path between them”), which is a property

of the graph itself, independent of any traversal. BFS from any vertex v in a component visits exactly the set of vertices reachable from v – which, for an undirected graph, is exactly v 's entire connected component (reachability is symmetric: if u is reachable from v , v is reachable from u by reversing the path). So regardless of which vertex of a component is visited first (or the BFS order within it), the *set* of vertices marked as belonging to that component is always the same: the full component. The outer loop over $v = 0, \dots, n - 1$ then assigns each component to whichever of its vertices has the smallest index, which is also well-defined and order-independent.

- A7.** Computing $\text{ecc}(v)$ via one BFS from v costs $O(n + m)$ (Question A4). Doing this for all n vertices costs $n \times O(n + m) = O(n(n + m)) = O(n^2 + nm)$. For $n = 10^6$ and even a sparse graph ($m = O(n)$), this is $O(n^2) = O(10^{12})$ operations – far too slow for any practical time budget. (Faster approximate or exact diameter algorithms exist but are beyond this week's scope.)
- A8.** (a) The maximum recursion depth of recursive DFS (and correspondingly the maximum size of the explicit stack in iterative DFS) is bounded by the length of the longest simple path explored, which in the worst case is $\Theta(n)$ – achieved on a *path graph* $0 - 1 - 2 - \dots - (n - 1)$, where DFS from vertex 0 recurses all the way down to vertex $n - 1$ before any frame returns. (b) Recursive DFS visits $\text{adj}[u][0]$ first, then (after that whole subtree finishes) $\text{adj}[u][1]$, etc. An explicit stack is LIFO: the *last* thing pushed is popped *first*. To have $\text{adj}[u][0]$ popped (and hence visited) first, it must be pushed *last* – i.e. the neighbors must be pushed in reverse order ($\text{adj}[u][k - 1], \dots, \text{adj}[u][1], \text{adj}[u][0]$), so that $\text{adj}[u][0]$ ends up on top of the stack.
- A9.** $d[u]$ is the value of a global counter at the moment u is first discovered (visited); $f[u]$ is the value of the (same, continuously incremented) counter at the moment the recursive call processing u returns (all of u 's descendants fully explored). **Parenthesis property:** for any two vertices u, v , the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are either completely disjoint, or one is entirely nested within the other – they can never partially overlap (e.g. $d[u] < d[v] < f[u] < f[v]$ is impossible).
- A10.** ($\Rightarrow$) If DFS finds such an edge (u, v) (a back edge to a visited, non-parent vertex v), then v was visited *before* u in this DFS, so there is a tree path from v down to u (since v is an ancestor of u – if v were in a different, already-finished subtree, it would have to be *black*/finished, but in an undirected graph all back edges point to ancestors). This tree path $v \rightarrow \dots \rightarrow u$, together with the edge (u, v) , forms a cycle. ($\Leftarrow$) If the graph has a cycle $v_0 - v_1 - \dots - v_{k-1} - v_0$, let v_i be the first vertex of the cycle visited by DFS. DFS will eventually traverse the cycle from v_i in one direction, say reaching v_{i-1} (the other cycle-neighbor of v_i) via the path through all other cycle vertices; when DFS is at v_{i-1} and examines the edge to v_i , v_i is already visited and is *not* v_{i-1} 's parent (since v_{i-1} 's parent is the previous vertex on this path, not v_i , for $k \geq 3$) – so this is exactly such a back edge.
- A11.** The function `edges_to_adjacency_list` (for undirected graphs) appends v to $\text{adj}[u]$ and u to $\text{adj}[v]$ for every edge (u, v) – this is necessary because an undirected edge $\{u, v\}$ means “ u and v are adjacent”, which must be reflected symmetrically in both adjacency lists. Consequently, when DFS is at u (having arrived via the edge from $\text{parent}[u]$), $\text{adj}[u]$ contains $\text{parent}[u]$ as one of its entries – this is simply the *same edge* traversed in the opposite direction, not a new edge forming a cycle. A naive detector that flags *any* visited neighbor (including the parent) as a cycle would report a cycle on every edge of every graph with ≥ 1 edge – even a single edge $\{0, 1\}$ would be flagged, since from 1, neighbor 0 (the parent) is already visited.

- A12.** A graph $G = (V, E)$ is **bipartite** if $V = A \cup B$ (disjoint) such that every edge has one endpoint in A and one in B . A **2-coloring** is a function $color : V \rightarrow \{0, 1\}$ such that $color(u) \neq color(v)$ for every edge $\{u, v\}$ (so $A = color^{-1}(0)$, $B = color^{-1}(1)$). **BFS algorithm:** for each unvisited vertex s (handling multiple components), set $color(s) = 0$ and run BFS; whenever a vertex v is discovered via u , set $color(v) = 1 - color(u)$. If BFS ever examines an edge $\{u, v\}$ where both u, v are already visited and $color(u) = color(v)$, the graph is not bipartite (return **None/False**). Different components are colored independently (each component's BFS starts fresh with $color = 0$ for its root), since bipartiteness is a per-component property and there is no constraint linking colors across components.
- A13. Theorem:** G is bipartite iff G has no odd cycle. $(\Rightarrow)$ If G is bipartite with parts A, B , consider any cycle $v_0 - v_1 - \dots - v_{k-1} - v_0$. Each edge $\{v_i, v_{i+1}\}$ goes between A and B , so $color(v_i) \neq color(v_{i+1})$ for all i (indices mod k). This means the colors alternate around the cycle; for the cycle to close consistently ($color(v_0)$ must equal itself after k alternations), k must be even. So every cycle has even length – there is no odd cycle. $(\Leftarrow)$ Suppose G has no odd cycle. 2-color each component via BFS from a root s : set $color(v) = dist(s, v) \bmod 2$. Suppose, for contradiction, some edge $\{u, v\}$ has $color(u) = color(v)$, i.e. $dist(s, u) \equiv dist(s, v) \pmod{2}$. The BFS-tree path $s \rightarrow u$ (length $dist(s, u)$), the edge $\{u, v\}$ (length 1), and the BFS-tree path $v \rightarrow s$ (length $dist(s, v)$) together form a closed walk of length $dist(s, u) + dist(s, v) + 1$, which is *odd* (sum of two same-parity numbers is even, plus 1). This closed walk contains a cycle of odd length (the walk may not be simple, but any closed walk of odd length contains an odd simple cycle), contradicting the assumption. So no such edge exists, and the BFS coloring is a valid 2-coloring – G is bipartite.
- A14.** Let u be the deeper-or-equal vertex in the BFS tree rooted at s (i.e. $d_u = dist(s, u) \geq d_v = dist(s, v)$), where the same-color edge is $\{u, v\}$. The desired odd cycle is formed by: the BFS-tree path from s down to u (length d_u), the edge $\{u, v\}$ (length 1), and the BFS-tree path from v back up to s (length d_v). Concretely, reconstruct each path by following parent pointers from u (resp. v) up to the root s , then splice: $[s \rightarrow \dots \rightarrow u] + [u, v] + [v \rightarrow \dots \rightarrow s]$, removing the duplicated occurrence of s at the seam (or, more simply, find the lowest common ancestor of u, v in the BFS tree and combine the two upward paths from u and v to it plus the edge). The total length is $d_u + d_v + 1$. Since $color(u) = color(v)$ means $d_u \equiv d_v \pmod{2}$ (as $color(\cdot) = dist(s, \cdot) \bmod 2$), $d_u + d_v$ is even, so $d_u + d_v + 1$ is odd.
- A15. White:** the vertex has not yet been discovered by DFS. **Gray:** the vertex has been discovered and is currently “open” – it is on the path from the DFS root to the vertex currently being explored (an ancestor of, or equal to, the current vertex in the recursion stack). **Black:** the vertex's recursive DFS call has returned – all vertices reachable from it (in the DFS tree) have been fully explored.
- A16.** $(\Rightarrow)$ If DFS examines edge (u, v) with v gray, then v is on the current recursion stack, meaning v 's recursive call has not yet returned and u is being explored as part of (a descendant of) that call – i.e. v is an ancestor of u in the DFS tree, so there is a directed tree-path $v \rightarrow \dots \rightarrow u$. The edge (u, v) closes this path into a directed cycle $v \rightarrow \dots \rightarrow u \rightarrow v$. $(\Leftarrow)$ Suppose G has a directed cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k-1} \rightarrow v_0$. Let v_i be the first vertex of this cycle discovered by DFS (colored gray first among them). At that moment, no other v_j ($j \neq i$) has been discovered yet (if some v_j had been discovered earlier, DFS would have to have reached v_i via the cycle from v_j , contradicting v_i being first – more carefully: if any other cycle vertex were discovered first, it would eventually either reach v_i along the cycle while still gray, giving

a back edge there instead, or v_i would be discovered as a descendant). DFS from v_i proceeds along $v_i \rightarrow v_{i+1} \rightarrow \dots \rightarrow v_{i-1}$ (the cycle, relabeled); when the edge (v_{i-1}, v_i) is examined, v_i 's call has not yet returned (it is still an ancestor of v_{i-1} in the active recursion), so v_i is gray – this is the required back edge.

- A17.** A **topological order** of a DAG $G = (V, E)$ is a sequence listing all vertices of V such that for every edge $(u, v) \in E$, u appears before v in the sequence. ($\Rightarrow$, **has order** $\Rightarrow$ **acyclic**): suppose G has a cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k-1} \rightarrow v_0$ and also a topological order. The order requires v_0 before v_1 , v_1 before v_2 , $\dots$, v_{k-1} before v_0 – chaining these gives v_0 before v_0 , a contradiction. So a graph with a topological order must be acyclic. ($\Leftarrow$, **acyclic** $\Rightarrow$ **has order**): by induction on $n = |V|$. If $n = 0$, the empty order works. Otherwise, a nonempty DAG must have at least one vertex with in-degree 0 (a “source”): otherwise every vertex has an incoming edge, and following incoming edges backward indefinitely from any vertex must eventually repeat a vertex (finite graph), producing a cycle, contradicting acyclicity. Pick such a source u , place it first, and recurse on $G - u$ (still a DAG, with $n - 1$ vertices) to get a topological order of the rest; prepending u gives a valid order for G (every edge out of u goes to a vertex placed after u ; all other edges are among $G - u$'s vertices and respected by the recursive order).
- A18.** Consider any edge $(u, v) \in E$. We must show $f[v] < f[u]$ (so that, in the reversed-finish order, u comes before v). When DFS examines edge (u, v) , three cases for v 's color: **white** – DFS recurses into v immediately, and since u 's call cannot return until this recursive call (and everything it spawns) returns, v finishes strictly before u does, i.e. $f[v] < f[u]$. **black** – v has already finished, so $f[v]$ was already set before u examined this edge, hence certainly $f[v] < f[u]$ (since u has not finished yet). **gray** – this would mean v is an ancestor of u on the recursion stack, so the tree path $v \rightarrow \dots \rightarrow u$ plus the edge (u, v) forms a directed cycle – *this is exactly where the DAG/acyclicity assumption is used*: in a DAG this case cannot occur. So in the only two possible cases, $f[v] < f[u]$ for every edge (u, v) , which is exactly the condition for the reverse-finish order to be a valid topological order.
- A19.** **Kahn's algorithm**: compute $\text{indeg}[v]$ for every v (number of edges (u, v)); initialize a queue with all vertices of in-degree 0; repeatedly dequeue a vertex u , append it to the output, and for each out-neighbor v of u , decrement $\text{indeg}[v]$ – if it becomes 0, enqueue v . **Correspondence**: $\text{indeg}[v] = 0$ (among vertices not yet placed) means every edge (w, v) has its source w already placed in the output – i.e. the topological-order constraint “ w before v ” is already satisfied for all such w , so placing v next violates nothing. Conversely, if $\text{indeg}[v] > 0$, some unplaced w with $(w, v) \in E$ must come before v , so v cannot legally be placed yet. If the algorithm terminates having output $< n$ vertices, the remaining vertices all have positive in-degree from *each other* (every edge among them points to a vertex still “blocked”), which means the remaining vertices form a sub-digraph where every vertex has an incoming edge from within the set – exactly the condition that forces a cycle (Question A17's argument), so the original graph is *not* a DAG.
- A20.** **Two topological orders**: G with $V = \{0, 1, 2, 3\}$ and edges $0 \rightarrow 2, 1 \rightarrow 3$ (two independent chains of length 1). Both $[0, 1, 2, 3]$ and $[1, 0, 2, 3]$ (and others) are valid, since there is no edge constraining the relative order of 0 and 1, nor of 2 and 3, etc. **Unique topological order**: G with $V = \{0, 1, 2, 3\}$ and edges $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3$ (a single chain / total order) – the only valid order is $[0, 1, 2, 3]$. **Structural property**: a DAG has a unique topological order iff every pair of consecutive vertices in that order is directly connected by an edge –

equivalently, the DAG contains a **Hamiltonian path** consistent with E (a path visiting every vertex exactly once, following edges). If any two “adjacent in the order” vertices lack a direct edge, they could be swapped (if no other constraint forbids it) or, more generally, the absence of a total ordering of constraints leaves freedom.

A21. Quick-Find: array $id[0..n-1]$, initially $id[i] = i$. $find(p) = id[p]$, $O(1)$. $union(p, q)$: if $id[p] \neq id[q]$, scan *all* n entries and set $id[i] \leftarrow id[q]$ for every i with $id[i] = id[p]$ – $O(n)$. **Proof of $\Theta(n^2)$:** consider the sequence $union(0, 1), union(1, 2), \dots, union(n-2, n-1)$ (each union merges the singleton/growing component containing the new element into the existing one). The k -th union (for $k = 1, \dots, n-1$) merges a component of size k (elements $0, \dots, k-1$, all sharing id after previous unions) with the singleton $\{k\}$; relabeling the smaller-or-equal one of these requires touching $\Theta(k)$ entries (e.g. relabeling the k -element component to match element k ’s id). Summing over $k = 1$ to $n-1$ gives $\sum_{k=1}^{n-1} \Theta(k) = \Theta(n^2)$.

A22. Quick-Union: array $parent[0..n-1]$, initially $parent[i] = i$ (each element is its own root). $find(p)$ follows $parent$ pointers until reaching a self-parent (root) – $O(\text{height})$. $union(p, q)$ sets $parent[find(p)] \leftarrow find(q)$ – $O(\text{height})$. **Path-producing sequence:** $union(1, 0), union(2, 1), union(3, 2), \dots, union(n-2, n-1)$. After $union(1, 0)$: $parent[1] = 0$ (root of $\{0, 1\}$ is 0). After $union(2, 1)$: $find(1) = 0$, so $parent[0] \leftarrow find(2) = 2$ – giving $parent[0] = 2, parent[1] = 0$, i.e. a chain $1 \rightarrow 0 \rightarrow 2$ with root 2. Continuing this pattern (each union attaches the current root under the new element), after all $n-1$ unions the **parent** pointers form a path of length $n-1$ with the most recently unioned element as the root. A subsequent $find$ on the original deepest node (element 0 or 1, deep in this chain) must walk all $\Theta(n)$ pointers to reach the root – $\Theta(n)$ time for a single $find$.

A23. Union by rank: each root tracks a *rank* (initially 0); $union(p, q)$ attaches the root of the *smaller-rank* tree under the root of the larger-rank tree (ties broken arbitrarily, incrementing the surviving root’s rank by 1 only when ranks were equal). **(a) Proof of $O(\log n)$ height:** by induction, a tree with rank r contains at least 2^r nodes. Base case $r = 0$: a single node, $2^0 = 1$. Inductive step: merging two trees with ranks $r_1 \leq r_2$ produces a tree of rank $\max(r_1, r_2)$ (if $r_1 < r_2$) or $r_1 + 1$ (if $r_1 = r_2$), and node count $\geq 2^{r_1} + 2^{r_2} \geq 2 \cdot 2^{\min(r_1, r_2)} = 2^{\min(r_1, r_2)+1}$, which is $\geq 2^{\text{new rank}}$ in both cases. Since the whole structure has n nodes, any tree’s rank r satisfies $2^r \leq n$, i.e. $r \leq \log_2 n$; and the height of a tree is bounded by its rank, so $find$ costs $O(\log n)$. **(b) Path compression intuition:** path compression makes every node visited during a $find$ point directly to the root afterward, so future $find$ s through those nodes become $O(1)$. Combined with union-by-rank’s $O(\log n)$ height bound, the *total* “flattening work” across a long sequence of m operations is bounded by a slowly-growing function of n (the inverse Ackermann function $\alpha(n)$, which is ≤ 4 for any n up to astronomically large values), so the *amortized* cost per operation – total work divided by number of operations – is $O(\alpha(n))$, effectively constant in practice.

Part B

B1. True. BFS processes vertices in increasing order of $dist(s, \cdot)$, because of the FIFO queue and the layer-by-layer argument of Theorem (BFS correctness): all distance- k vertices are enqueued before any distance- $(k+1)$ vertex.

B2. (b) $O(n+m)$. Each vertex is processed once and each edge is examined a constant number of times (Question A4).

- B3. 2.** A self-loop (v, v) contributes 2 to $\deg(v)$, so that $\sum_v \deg(v) = 2|E|$ holds even when self-loops are present.
- B4. False.** The edge $(u, \text{parent}[u])$ in an undirected graph is simply the same edge traversed backward (the one DFS arrived on), not a cycle. Only a visited, *non*-parent neighbor indicates a cycle.
- B5. (c) no odd cycle.** This is the bipartite / odd-cycle characterization (Question A13).
- B6.** An edge to a **black** vertex indicates **no cycle** (it is a forward or cross edge – v is already fully processed and cannot be an ancestor of the current vertex). An edge to a **gray** vertex indicates a **cycle** (a back edge to an ancestor).
- B7. True.** Every DAG has at least one topological order (Question A17). If two vertices u, v have no edge between them and (more precisely) no directed path between them in either direction, then any valid topological order $\dots, u, \dots, v, \dots$ (WLOG u before v) can be modified by moving v to immediately before u (or any position not violating other constraints): since nothing forces u before v or v before u , both relative orderings are consistent with all edge constraints, giving at least two distinct valid topological orders.
- B8. (b) BFS with a queue, using in-degrees.** Kahn’s algorithm mirrors BFS: it uses a queue and processes vertices whose “in-degree among remaining vertices” has dropped to 0, analogous to BFS discovering vertices at the next distance layer.
- B9.** With Quick-Find, a single **union** costs $O(n)$ worst case (relabeling up to all n entries); a single **find** costs $O(1)$ (direct array lookup).
- B10. False.** The tightest known bound for union by rank + path compression is $O(\alpha(n))$ amortized (inverse Ackermann), which is asymptotically *much smaller* than $O(\log n)$ – for all practical n , $\alpha(n) \leq 4$, i.e. effectively $O(1)$.